

Упражнение 4 – Методологии за разработка на софтуер

Цел на упражнението: Основната цел на четвъртото упражнение е запознаване на студентите с различни методологии за софтуерна разработка. Упражнението няма за цел да даде изчерпателен детайлен анализ на всяка от разглежданите методологии. На база на предоставената информация студентите трябва да направят изводи за случаите на употреба, предимствата и недостатъците за всяка от разглежданите методологии.

Ключови думи: методологии за разработка на софтуер; software development methodologies; hacking; waterfall; evolutionary model; еволюционен модел; agile методологии, гъвкави методологии; kanban; канбан; scrum; скръм; extreme programming (XP).

Съдържание

1. Методологии за разработка на софтуер [1].....	1
2. Hacking[1].....	4
3. Waterfall[1].....	4
4. Evolutionary model (Еволюционен модел)[1].....	4
5. Agile методологии[1].....	5
6. Kanban[1].....	5
7. Scrum[1].....	5
8. Lean [3][4].....	7
9. Екстремно програмиране, Extreme Programming, XP.....	7
10. Задачи за изпълнение.....	8
Задача 1.....	8
Задача 2.....	8
Задача 3.....	8
Задача 4.....	9
Задача 5.....	9
Задача 6.....	9
Задача 7.....	9

1. Методологии за разработка на софтуер [1]

Съществуват множество подходи при създаването на софтуер. Основните разлики между всички тях са в прилаганите концепции за всеки от етапите от жизнения цикъл на софтуерната разработка.

Бележка: Припомняне на етапите от жизнения цикъл

Етапи от жизнения цикъл:

- Основен (главен) етап:
 - поддръжка - Този процес започва когато системата трябва да бъде модифицирана поради грешка, недостиг, проблем или необходимост от адаптация или добавяне на нови функционалности. Процесът завършва тогава, когато програмата “остарява” и става неизползваема [2];
 - реално функциониране - Този процес се състои в оперирането със софтуера и поддръжката му, т.е. използването на продукта от квалифицирания (обучения) оператор [2];
 - разработка - Най-общо този процес може да се раздели на два подпроцеса:

- да се разработват прототипи за изучаване на изискванията и дизайна на продукта;
 - да се произведе продукта [2].
- доставяне - Този процес се дефинира с дейности и задължения на доставчика. Процесът започва от решението да се отговори на запитването на клиента за изготвяне на софтуер. Може за начало на този етап да се вземе и подписването или постигането до договореност с клиента [2];
 - Поставяне на задачата за изработка на софтуер - Този процес се дефинира с дейности и задължения на поръчителя на софтуера. Задължение на поръчителя е да формулира ясно задание (техническо задание) към изпълнителя на поръчката. По това поставено задание изпълнителят трябва да даде информация на поръчителя за всички детайли по изработката на поръчката. Допълнително задължение на поръчителя на софтуера е да следи за изпълнението на поставеното задание – качество на изпълнението, реализирани функционалности, желан интерфейс, и др. [2]. За да се осъществява качествен мониторинг има много подходи, например да се изискват демонстрации с прототипи на системата, честа комуникация между поръчител и изпълнител и др.
- Етап на поддръжка:
 - документиране - Това е процес на съхраняване на информацията за целия процес на създаване на софтуера. Документацията описва дейностите по планирането, дизайна, разработката, корекциите и тестването на изисквания проект. Поддържането на тази документация актуална е важна за мениджърите по проекта, инженерите, които го изпълняват, и крайните потребители на системата [2];
 - управление на конфигурацията - Този процес се използва за [2]:
 - идентифициране и дефиниране на отделните елементи на системата;
 - контролиране на промените и реализацията на елементите;
 - записване и отчитане на статуса на всеки от елементите, заедно с плановете и заявките за коригиране на съответния елемент.
 - гарантиране на качеството - В края си, процесът предоставя фреймуърк за независимо и обективно подsigуряване на клиента. Клиентът трябва да получи гаранция, че предоставения му продукт, осигурява функционалностите, договорени в началото, и е изпълнен в договорения срок [2];
 - потвърждение (верификация) - Верификацията определя дали изискванията към системата са изпълнени според поставената задача. Този процес включва верификацията на целия процес по създаването на даден софтуер – изисквания, дизайн, код, интегриране, документация. Верификацията може да се изпълни от клиента, изпълнителя на задачата или от трети независими лица [2];
 - валидация - Валидацията определя дали крайната (реализираната вече) система, изпълнява своите специфични задължения. В този процес се включва няколкократно тестване на устойчивостта на разработената система. Валидирането може да се приеме като част от оценяването на даден проект или система. Степента на валидирането се определя от критичността към проекта. Валидацията може да се изпълни от клиента, изпълнителя на задачата или от трети независими лица [2].
 - съвместен преглед - Този процес предоставя фреймуърк за общуване (контактуване) на поръчителя и изпълнителя. По този начин двете страни споделят информация и мнение, коментират и доуточняват техническото задание, планират реализацията на проекта и цената на реализираните функционалности [2];

- проверка - Този процес предоставя фреймворк за официални, описани в споразумението, проверки на услугата, която трябва да се изпълни. Дадена проверка може да се ръководи от клиента (възложителя на задачата). При една такава проверка, проверяващия трябва да даде оценка на създадените до момента продукти или изпълнените до момента услуги. Неговото внимание трябва да бъде насочено към това дали се спазват предвидените срокове и дали се изпълняват предварителните договори, описващи изискваните от системата функционалности [2];
- разрешаване на проблема - Този процес предоставя механизъм за затваряне на цикъла на производството на софтуера. Състои се в това, че при сигнализиране за даден проблем, се прави подробен анализ на причините за възникването на проблема, предприемат се адекватни и правилни действия за отстраняването на проблема и се дава информация на клиента за същността на проблема (осигурява се обратна връзка). Тук под проблем трябва да се разбира не само грешка в работата на софтуера, но и всяко несъответствие между техническото задание и реализирания софтуер [2], за което е било или ще бъде заплатено.
- Организационен етап:
 - управление - Този процес определя основните дейности и задължения на мениджъра на проекта. Той включва инициатива за създаване на проект, формулиране на техническо задание, планиране, изпълнение и контрол, преглед, оценка и финализиране на проекта [2];
 - инфраструктура - Под инфраструктура в случая може да се разбира хардуер, софтуер, стандарти, инструменти, техники, апаратура, т.е. всичко, от което може да зависи реализацията на дадено задание. Този процес определя дейностите, необходими за установяване и поддържане на функционираща инфраструктура по време на жизнения цикъл на проекта. Процесът включва: имплементация, създаване на инфраструктура и поддръжка на инфраструктурата;
 - подобряване - Стандартът предоставя основните дейности, които една организация трябва да оцени, премери, контролира и подобри. Дейностите на разработчиците включват: установяване, оценяване и подобряване на даден процес. Организацията трупа опит от прилагането на процесите върху различни проекти. На базата на натрупания опит организацията подобрява процесите като цяло в организацията на текущи и бъдещи проекти;
 - обучение - Този процес се използва при подготовката на работна ръка за работа със разработения софтуер. Операторите със този софтуер трябва да придобият умения в управлението на софтуера. Процесът изисква да се изготви тренировъчен план, да се генерират обучителни материали и да се предостави обучение на служителите, които трябва да оперират със създадения софтуер.

Класификации за видове методологии:

- според гъвкавостта:
 - адаптивни - залагат на бързото адаптиране към променящата се реалност;
 - планиращи - обръщат особено внимание на процеса на планиране;
 - формални - разчитат на различни подходи за формална проверка за коректност.
- според броя итерации:
 - Традиционни;
 - Итеративни.
- според задълбочеността на планирането:

- Тежки методологии (Heavy Methodologies, plan driven методологии);
- Леки методологии (Light Methodologies).

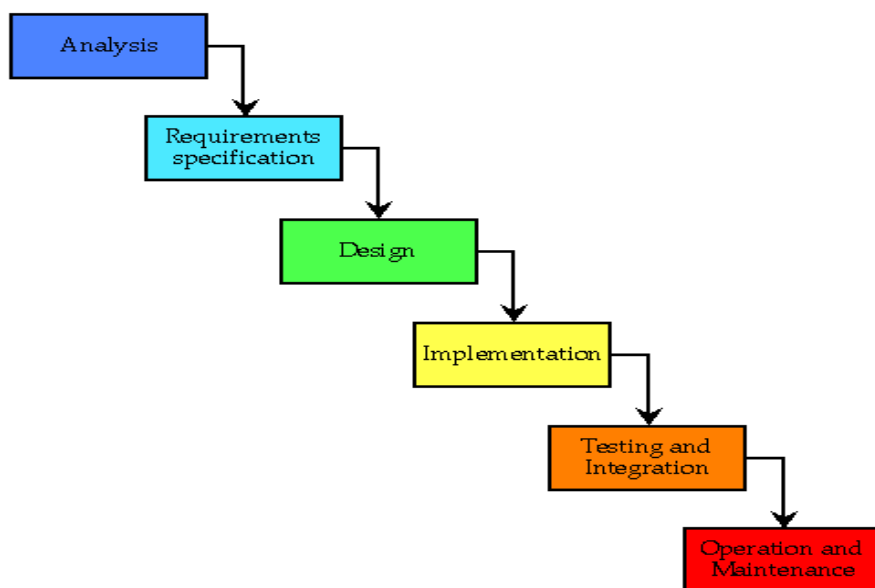
2. Hacking[1]

Прилагането на hacking подхода не включва предварително разбиране или документиране на потребителските желания, техническите изисквания, описание на структурата на проекта или планиране на тестове. Hacking моделът кара разработчиците да мислят за детайлите в софтуерната разработка, вместо да помислят първо за проблеми от по-високо ниво. Разработчикът едновременно се опитва да си обясни изискванията, структурата, съображенията за тестването директно чрез синтаксиса на програмния език.

3. Waterfall[1]

През 1970г. Уинстън Ройс представя няколко модела за разработка на софтуер в своята книга, един от които е методът на водопада.

Основната задача на модела на водопада е разработката да се представи като систематизиран, последователен процес от фази. Всяка фаза завършва с отчет на важните събития (milestone review). От Фигура 1 (графиката на модела) лесно се забелязва, че идеята е да се преминава гладко от фаза във фаза в прогресия, точно както водата тече надолу по водопад, като преди преминаването към следващата фаза, единствено ако предходната фаза е приключила и са изготвени нужните отчети.



Фигура 1: Waterfall model

4. Evolutionary model (Еволюционен модел)[1]

Еволюционният модел се свързва с очаквани промени в изискванията, структурата и кода. Самата разработка се съобразява с идеята, че ще има следващ етап на доработка на предходната версия. Този модел е предпочитан в случаи, в които изискванията и софтуерната архитектура не могат да бъдат специфицирани изцяло предварително. Полезно е да се използва за нови типове проекти, с които фирмата няма опит.

5. Agile методологии[1]

Гъвката методология за разработка на софтуер представлява адаптивен подход за разработка на софтуер. От друга страна гъвката методология представлява група от методи за разработка на софтуер. Тези методи се основават на непрестанно развитие, където изискванията и решенията се променят чрез сътрудничество между самоорганизиращи се екипи. Терминът се формулира през 2001 година при формирането на Манифеста на гъвката методология.

Гъвката методология насърчава адаптивно планиране, еволюираща разработка и доставяне на софтуера, времево-разпределен итеративен подход и поощрява бързото и гъвкаво реагиране на промени. Скоро след своето създаване, тя революционизира модерното програмиране, превръщайки се в модел за множество разработчици на софтуер по цял свят.

Някои от най-известните гъвкави методи са:

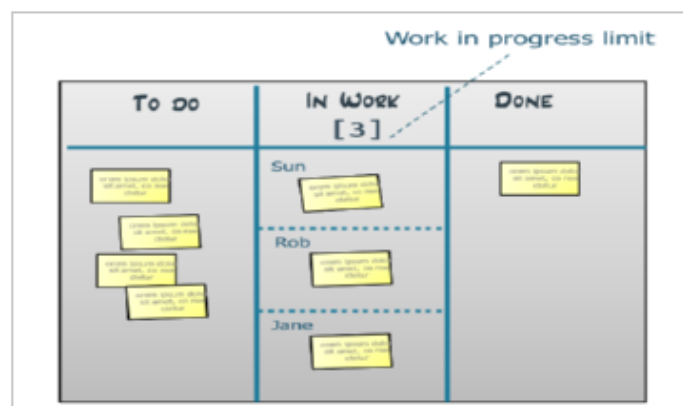
- Канбан (Kanban) – т. 6 Kanban[1], стр. 5;
- Скръм (Scrum) – т. 7 Scrum[1], стр. 5;
- Лийн (Lean) – т. 8 Lean [3][4], стр. 7;
- Екстремно програмиране (XP) – т. 9 Екстремно програмиране, Extreme Programming, XP, стр. 7.

6. Kanban[1]

Канбан е метод за управление на интелектуалната дейност, който набляга на доставката точно навреме, без същевременно членовете на екипа да са пренатоварени. При този подход процесът - от дефинирането на задачата до доставянето на продукта, е изложен на показ за участниците в него, а същевременно разработчиците на софтуера взимат работни задачи от така наречената „опашка“.

Моделът използва система, при която се ограничават количеството на текуща работа във всеки един момент от работния процес, а допълнителни задачи могат да се поемат едва тогава, когато е възможно да има работен прогрес върху тях. Този подход е основен механизъм за разкриване на системни операционни проблеми и стимулиране на взаимопомощта, с цел продължително подобряване на системата.

На Фигура 2 е показана Канбан дъска. Там трябва да се забележи разделянето на производствения процес на стъпки – TODO, IN WORK и DONE. Важен елемент, който е една от основните характеристики на Канбан, е ограничението на броя задачи във всяка стъпка от разработката.



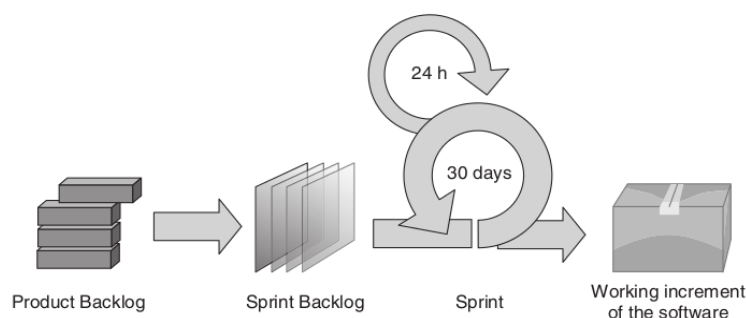
Фигура 2: Канбан дъска

7. Scrum[1]

Скръм е рамка за планиране и разработка на софтуерни проекти, базирана на принципите на

Гъвкавата методология за разработка на софтуер. Работата по Scrum методологията се свързва с определени термини:

- User Stories - Преведено на български език **User Stories** е „Приказки на потребителя“. В контекста на Scrum методологията терминът следва да се разбира описание на изискванията и ограниченията, поставени от потребителя;
- Product Backlog - **Product Backlog** е списък с неизпълнени задачи. За да се постави задача в Product Backlog, задачата трябва да е преминала през етапа на планиране. Обикновено задачите в Product Backlog са подредени по приоритет от project manager-а;
- ScrumMaster - **ScrumMaster** се нарича project manager-а или project leader-ът. ScrumMaster-ът описва изискванията на потребителя, подрежда ги по приоритет и ги поставя в списъка с неизпълнени задачи;
- Product Owner - Преведено на български език **Product Owner** е „собственик на продукта“. На практика собственик на продукта се явява клиентът;
- Team - Преведено на български език **Team** е „отбор“. В контекстът на Scrum методологията терминът назовава екипа от разработчици. Обикновено един Team съдържа до десетима разработчици;
- Sprint - **Sprint** представлява една итерация в разработката на софтуерния проект. Една итерация трае обикновено около 30 дни. В края ѝ има възможност готовите компоненти да се покажат на възложителя;
- Sprint planning meeting - **Sprint planning meeting** представлява среща за планиране на итерацията. По време на тези срещи се решава кои задачи от списъка с неизпълнени задачи (Product Backlog) преминават в списъка на задачите, които се изпълняват в текущата итерация (Sprint Backlog). Тези срещи обикновено траят около 15 минути и се провеждат всеки ден от итерацията. Целта на тези срещи е да се обсъди свършената работа от предходния ден и да се планира работата за текущия ден. Тези срещи позволяват на project manager-а да определи задачите, които трябва да бъдат завършени през деня и да укаже съдействие при възникнали проблеми от предишния ден. Чрез тези срещи обикновено уменията на project manager съумява да предвиди рисковите фактори и открива потенциални проблеми;
- Sprint Backlog - **Sprint Backlog** представлява списък от задачи, които се изпълняват в текущата итерация. За да се постави задача в този списък, то задължително преди това се преминава през sprint planning meeting;
- **Sprint Retrospective** - **Sprint Retrospective** представлява среща, на която екипът от разработчици прави преглед на изминалата итерация. Това е среща, която се състои непосредствено след приключване на всяка итерация.



Фигура 3: Scrum

На Фигура 3 е показана схемата на процесите при софтуерната разработка по методологията Scrum. В началото ScrumMaster-ът след среща с Product Owner описва User Stories, разделя задачите, подрежда ги по приоритет и ги поставя в Product Backlog. Дава се начало на първи Sprint, но всеки от Sprint-овете протича по един и същ начин. Всеки ден от изпълнението на Sprint-а се състои Sprint planning meeting, където Team-ът обсъжда свършената работа от предходния ден и се дискутират задачите, които трябва да се свършат в текущия ден. Задачите, които ще се изпълняват, се местят в Sprint Backlog. В края на

Sprint-a се състои Sprint Retrospective, където Team-ът прави обзор на свършената работа по време на текущия Sprint. Ако Product Owner-ът прояви желание, може да види получените софтуерни компоненти след края на Sprint-a.

8. Lean [3][4]

Lean представлява адаптивна методология за софтуерна разработка, възприела основните принципи на системата за производство на Тойота.

Основната цел на Lean методологията е да създаде стойност за потребителите на даден бизнес и в същото време да намали разходите.

Lean е управленски подход, който насочва и ограничава разхода на производствените ресурси единствено и само в постигане на цели, които създават добавена стойност за крайния клиент.

Често фирмите прахосват ресурси за постигане на цели, които не създават добавена стойност. Така те понасят загуби, които рефлектират в ниска производителност, утежнена себестойност и нестабилно качество. Това затруднява системата за управление и тя допуска грешки, а те водят до още загуби и до още проблеми с производителността, себестойността и качеството. Фирмите влизат в омагьосан кръг – вместо да създават добавена стойност за клиента, те се борят с проблеми, които може и да са предизвикателни, но не правят продукта по-продаваем.

Бележка: Toyota Production System

От Toyota публикуват официално описание на TPS за първи път през 1992 година. Там дават следното описание: „TPS е фреймуърк, за съхранение на ресурси чрез премахване на загуби. Хората, които участват в прилагането на тази система, трябва да се научат да идентифицират разходите за материали, усилия и време, които не генерират стойност за клиентите. “. Основни принципи:

- Just-in-time – Стратегия да се прави само това, което е нужно, тогава, когато е нужно, и толкова (като количество), колкото е нужно;
- Jidoka - (Autopomation) - Автоматизация с намесата на човек. Състои се в автоматично откриване на аномално поведение и намесата на човек за спиране на производствената линия.

9. Екстремно програмиране, Extreme Programming, XP

Кент Бек дава определение на това какво представлява Екстрмното програмиране по следният начин [5]:

- Вземете ефективните практики при наблюдаването на екипното разработване на софтуер.
- Изпълнете ги в най-големите им крайности.

Това може да се изобрази с Таблица 1:

Добри практики	Практиките, изпълнени в крайност
Преглед на кода	Програмиране по двойки
Тестване	TDD (Test driven development)
Дизайн на софтуера	Изключително много рефакторинг
Простота	Най-простото нещо, което може да работи
Тестване при внедряването	Продължително внедряване
Кратки итерации	Планиране за всяка итерация

Таблица 1: Добри практики в разработката на софтуер и техния аналог в ХР

Основната цел на ХР методологията е да редуцира цената на проект, ако се наложи дадена промяна. Това го прави предпочитан метод за проекти, за които биха били необходими промени в бъдеще, тъй като стандартните методологии биха били скъспотруващи за реализацията им. Използват се при по-малки и по-лесни за реализация проекти (Бързо натрупване на средства)

10. Задачи за изпълнение

Задача 1

Изведете плюсове и недостатъци за всяка от разгледаните методологии за софтуерна разработка. В кои случаи е подходящо да се използват различните подходи?

Насоки:

- Време за реализация на проекта;
- Време за документиране на проекта;
- Време за реагиране на необходимост от промяна;
- Свобода на разработчиците;
- Лекота за работа в екип.

Задача 2

В контекста на Hacking методологията:

- До какво води кодирането без документация?
- До какво води кодирането без план?
- До какво води програмирането в среда без налагани ограничения от project manager-a?

Задача 3

В контекста на Waterfall методологията - Минус ли е елиминирането на творческия процес?

- Как ще повлияе подобен ход върху риска от грешки?
- Как ще се отрази на високо- и нискоквалифицираните програмисти?
- Как ще се отрази на креативните личности?

Задача 4

За всяка от разгледаните методологии да се опише фирма, за която прилагането на методологията, ще е успешно.

Насока: Хубаво е да се вземе предвид, че не е от значение само фирмата, а и конкретният проект, по който фирмата ще работи.

Задача 5

Опитайте се да определите прилагането на коя от методологиите би довело най-бързо до реализирането на даден проект. Защо?

Насока: Да се обърне внимание на времето, което се отделя за реализирането на всяка стъпка от жизнения цикъл на проект, при използването на конкретна методология.

Задача 6

Подходящо ли е в една фирма да се практикуват няколко методологии за разработка на софтуер, като за всеки проект се взема решение коя от тях да се практикува?

Насоки:

- Достатъчна ли е една методология за една фирма?
- Съвместими ли са всички методологии?
- Възможно ли е едни и същи разработчици да работят по различни методологии в различните си проекти?
- Ако един екип е специализиран в правенето на мобилни приложения и работи по методология от Agile, то с какъв успех екипът ще разработи информационна система по същата Agile методология?
- Едно от нефункционалните изисквания към софтуера може да бъде използването на конкретна методология за разработката. Ако една фирма практикува само една методология, то какво би направила фирмата, в случай че възложителят постави изискване за методологията и тя не съвпада с методологията, използвана от фирмата.

Задача 7

Подходящо ли е софтуерна фирма да използва основни характеристики от няколко методологии, за да изгради своя собствена методология-хибрид?

Насоки:

- Кой трябва да определи кои са „добрите“ черти на методологиите, от които се заимства? Собственикът на фирмата или project manager-а?
- Какви са начините, по които членовете на екипа да разберат същността на новата методология-хибрид? Дори и да не е избрана методология-хибрид, то какви са начините, за качествено разпространение на идеи сред колектива?

Използвана литература

- 1: RICHARD FAIRLEY, MANAGING AND LEADING SOFTWARE PROJECTS,
- 2: ISO/IEC 12207, INTERNATIONAL STANDARD ISO/IEC 12207 - SOFTWARE LIFE CYCLE PROCESSES, , <http://www.abelia.com/docs/12207cpt.pdf>

- 3: Unknown, Lean разработка на софтуер, , https://bg.wikipedia.org/wiki/Lean_%D1%80%D0%B0%D0%B7%D1%80%D0%B0%D0%B1%D0%BE%D1%82%D0%BA%D0%B0_%D0%BD%D0%B0_%D1%81%D0%BE%D1%84%D1%82%D1%83%D0%B5%D1%80
- 4: Lean.bg Team, Lean Мисленее, , <http://lean.bg/wcm/lean-thinking/>
- 5: Monjurul Habib, Agile software development methodologies and how to apply them., 2013, <http://www.codeproject.com/Articles/604417/Agile-software-development-methodologies-and-how-t>